

02477 Bayesian Machine Learning - Exercise 1

```
In [1]: %matplotlib inline
import pylab as plt
import jax.numpy as jnp
import seaborn as snb

from scipy.stats import binom as binom_dist
from scipy.stats import beta as beta_dist
from scipy.special import beta as beta_fun

snb.set_style('darkgrid')
snb.set(font_scale=1.5)
plt.rcParams['lines.linewidth'] = 3
```

Maximum Likelihood Estimation

► Click to expand MLE derivation

Introduction and purpose of the first exercise

► Click to expand the introduction and purpose of the first exercise

Content

- Part 1: Maximum likelihood estimation
- Part 2: Bayesian inference
- Part 3: The functional form of Beta distributions
- Part 4: Computing summary statistics and probabilities using sampling
- Part 5: Application to A/B testing

Estimating the proportions using the binomial distribution

In this exercise, we will work with two common families of probabilities distributions: the **Binomial distribution** and the **Beta distribution**.

►  Summary of Binomial Distribution

►  Summary of Beta Distribution

The Binomial Distribution and Maximum Likelihood Estimation

► Click to expand the introduction to the Binomial distribution

The Beta Distribution as a Prior for θ

► Click to expand the introduction to the Beta distribution

The Beta-Binomial Model

► Click to expand the introduction to the Beta-binomial model

Part 1: Maximum Likelihood Estimation

Suppose we observe:

- $N = 7$ independent Bernoulli trials
- $y = 1$ success

Then our dataset is: $\mathcal{D} = \{N = 7, y = 1\}$

We want to estimate the **probability of success** $\theta \in [0, 1]$ using **maximum likelihood**.

Task 1.1

 **Plot the likelihood** $p(y | \theta)$ as a function of θ over the interval $[0, 1]$.

 **Goal:** Identify the **maximum likelihood estimate** (MLE) visually **and/or** numerically.

Hints

- You can **implement the likelihood manually** using:

$$p(y | \theta) = \binom{N}{y} \theta^y (1 - \theta)^{N-y}$$

- Or you can use **SciPy's built-in function**:

```
from scipy.stats import binom
binom.pmf(y, n, p)
```

When computing the likelihood, we assume that y and N are fixed, and we vary θ . So we write:

$$\mathcal{L}(\theta) = p(y | \theta) = p(y | N = 7, \theta) = \binom{7}{1} \theta^1 (1 - \theta)^{7-1}$$

```
In [2]: from scipy.stats import binom

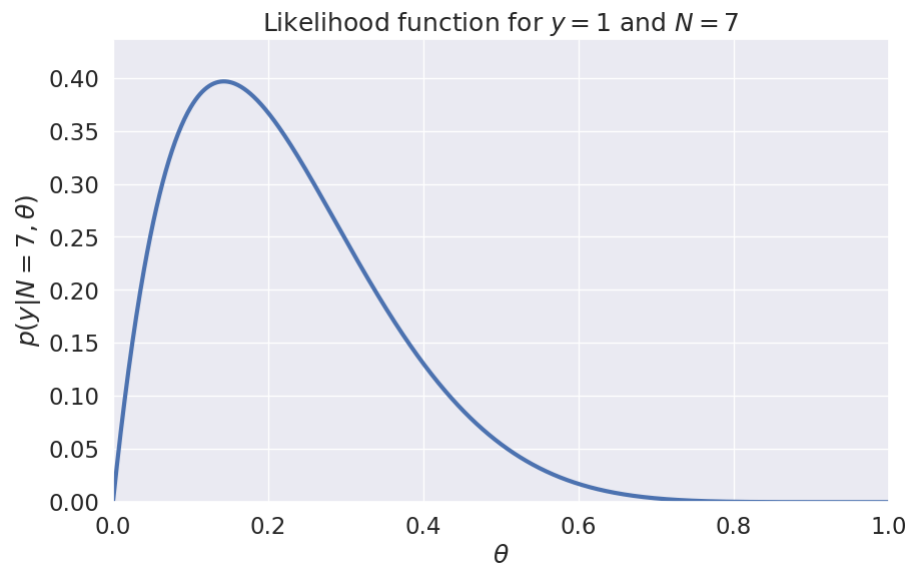
# data
N1 = 7
y1 = 1

# make grid for plotting the likelihood p(y|theta) in interval [0, 1]
thetas = jnp.linspace(0, 1, 1000)

def built_in_fun(y, n, p):
    return binom.pmf(y, n, p)

likelihood = built_in_fun(y1, N1, thetas)

fig, ax = plt.subplots(1,1, figsize=(10, 6))
ax.plot(thetas, likelihood)
plt.ylabel(r'$p(y|N=7, \theta)$')
plt.xlabel(r'$\theta$')
plt.title('Likelihood function for $y=1$ and $N=7$')
plt.xlim(0, 1)
plt.ylim(0, 1.1 * jnp.max(likelihood))
plt.show()
```



Task 1.2: Compute the maximum likelihood estimate for θ and compute a 95% confidence interval using the equations given above.

exercise 1.2

- using the equation $\hat{\theta}_{MLE} = \arg \max_{\theta} p(y|N, \theta)$

$\rightarrow$ which for the Binomial Distribution is $\hat{\theta}_{MLE} = \frac{y}{N}$, where $N=7$ and $y=1$

$$= \frac{1}{7} = \underline{\underline{0.14}}$$

95% confidence interval

$$\hat{\theta}_{MLE} \pm 1.96 \sqrt{\frac{\hat{\theta}_{MLE} (1 - \hat{\theta}_{MLE})}{N}}$$

$$= 0.14 + 1.96 \sqrt{\frac{0.14 (1 - 0.14)}{N}} = 0.39$$

$$= 0.14 - 1.96 \sqrt{\frac{0.14 (1 - 0.14)}{N}} = -0.11$$

```
In [3]: def MLE_estimate(y, N):
        return y/N

mle_binom = MLE_estimize(y1, N1)

print(f"The MLE estimate of the Binomial Distribution is {mle_binom:.3f}")

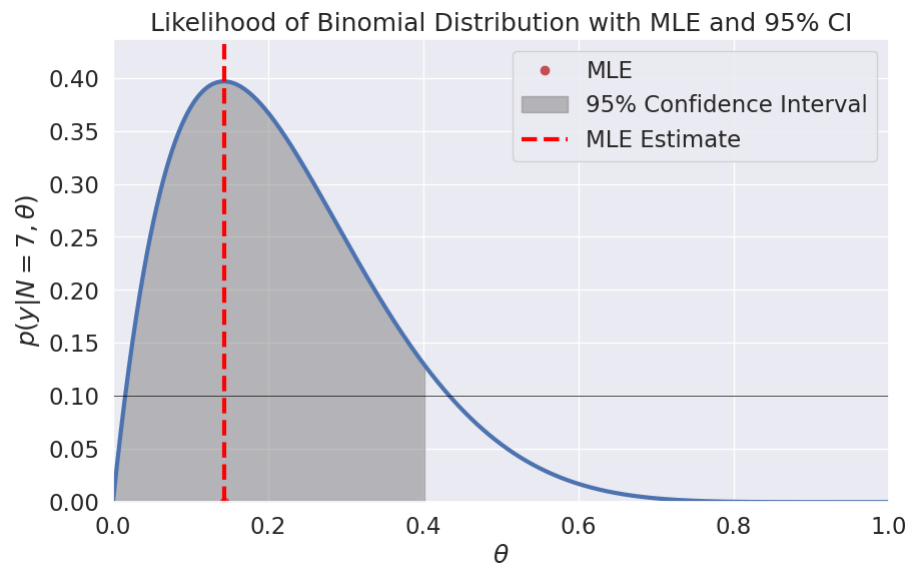
def conf95(MLE, N):
    fraction = 1.96 * jnp.sqrt(((MLE * (1 - MLE)) / (N)))
    return MLE + fraction, MLE - fraction

upper, lower = conf95(mle_binom, N1)

print(f"Upper 95: {upper:.3f}, Lower 95: {lower:.3f}")

fig, ax = plt.subplots(1,1, figsize=(10, 6))
ax.plot(thetas, likelihood)
ax.plot(mle_binom, likelihood[0], 'ro', label='MLE')
ax.fill_between(thetas, likelihood, where=(thetas >= lower) & (thetas <= upper),
               color='gray', alpha=0.5, label='95% Confidence Interval')
ax.legend()
plt.title('Likelihood of Binomial Distribution with MLE and 95% CI')
plt.xlim(0, 1)
plt.ylim(0, 1.1 * jnp.max(likelihood))
plt.axvline(mle_binom, color='red', linestyle='--', label='MLE Estimate')
plt.axhline(0, color='black', lw=0.5)
plt.axhline(1, color='black', lw=0.5)
plt.axhline(0.5, color='black', lw=0.5)
plt.axhline(0.1, color='black', lw=0.5)
plt.axhline(0.9, color='black', lw=0.5)
plt.legend()
plt.ylabel(r'$p(y|N=7, \theta)$')
plt.xlabel(r'$\theta$')
plt.show()
```

The MLE estimate of the Binomial Distribution is 0.143
Upper 95: 0.402, Lower 95: -0.116



Task 1.3: What happens if you had observed $y = 0$ instead of $y = 1$? Does the result seem reasonable?

Exercise 1.3

- What happens if we observe $y=0$, instead of $y=1$?

$$P(y=0|N=7, \theta) = \binom{N}{y} \theta^y (1-\theta)^{N-y}$$

$$= \binom{7}{0} 1 \cdot (1-\theta)^7$$

In [4]: `from scipy.stats import binom`

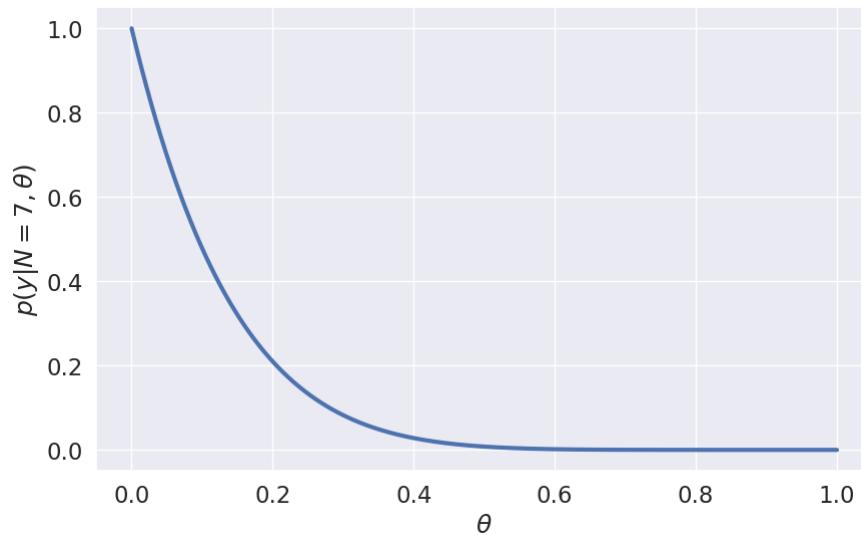
```
# data
N3 = 7
y3 = 0

# make grid for plotting the likelihood p(y|theta) in interval [0, 1]
thetas = jnp.linspace(0, 1, 1000)

def built_in_fun(y, n, p):
    return binom.pmf(y, n, p)

likelihood = built_in_fun(y3, N3, thetas)

fig, ax = plt.subplots(1,1, figsize=(10, 6))
ax.plot(thetas, likelihood)
plt.ylabel(r'$p(y|N=7, \theta)$')
plt.xlabel(r'$\theta$')
plt.show()
```



👉 We observe 0 successes out of 7 trials, which means that the MLE is at $\hat{\theta}_{MLE} = 0$.

- As θ increases, the probability of observing 0 successes becomes **smaller**, because we are assuming that success is more likely, but non occurred.
- The curve $\mathcal{L}(\theta) = (1 - \theta)^7$ is **strictly decreasing** in the interval $[0, 1]$.
- If no successes were observed out of 7 trials, the most likely explanation under the binomial model is that the probability of success θ is 0. As θ increases, the model expects more successes, but we have not observed any. Thus, the model assigns a lower probability to larger values of θ .

🎯 Part 2: Bayesian inference

We will now turn our attention towards Bayesian inference for θ . Recall, the core concept of Bayesian inference is that we infer a **full probability distribution** for θ rather than just a **point estimate** like $\hat{\theta}_{MLE}$. As before, your dataset is given by $\mathcal{D} = \{N = 7, y = 1\}$, but now we assume a **uniform prior distribution** for θ , i.e. $p(\theta) = \text{Beta}(\theta|a_0, b_0) = 1$ for $a_0 = b_0 = 1$.

📌 **Task 2.1:** Compute the **prior mean** and **variance** of θ , i.e. the mean and variance of $p(\theta)$.

Beta Prior

$$E[\theta] = \int_0^1 \theta p(\theta | a_0, b_0) d\theta = \frac{a_0}{a_0 + b_0}$$

$$V[\theta] = \int_0^1 (\theta - E[\theta])^2 p(\theta | a_0, b_0) d\theta = \frac{a_0 b_0}{(a_0 + b_0)^2 (a_0 + b_0 + 1)}$$

$$\theta_{mode} = \frac{a_0 - 1}{a_0 + b_0 - 2}$$

We assume Uniform Prior

Task 2.1: Compute prior mean and variance of $p(\theta)$, $a_0 = 1$, $b_0 = 1$

$$E[\theta] = \frac{a_0}{a_0 + b_0} = \frac{1}{1 + 1} = \frac{1}{2}$$

$$V[\theta] = \frac{a_0 b_0}{(a_0 + b_0)^2 (a_0 + b_0 + 1)} = \frac{1 \cdot 1}{(1 + 1)^2 (1 + 1 + 1)} = \frac{1}{12}$$

In [5]: # prior parameters

```
a_0 = 1
b_0 = 1
```

```
def prior_and_post_mean(a, b):
```

```
    """
    Compute the mean of a Beta distribution.
```

```
    Parameters
```

```
    a : float or int
```

```
        Shape parameter alpha of the Beta distribution (a > 0).
```

```
    b : float or int
```

```
        Shape parameter beta of the Beta distribution (b > 0).
```

```
    Returns
```

```
    mean : float
```

```
        The mean of the Beta(a, b) distribution.
```

```
    Equation
```

```
    mean = a / (a + b)
```

```
    Shapes
```

```
    a: scalar
```

```
    b: scalar
```

```
    returns: scalar
```

```
    Example
```

```
>>> prior_and_post_mean(1, 1)
```

```
0.5
```

```
>>> prior_and_post_mean(2, 3)
```

Task 2.2: Compute the parameters a and b of the posterior distribution, i.e. $p(\theta|y)$, using the equations for the Beta-binomial model.

Beta Binomial Posterior Distribution $P(\theta|y)$

Task 2.2 Compute parameters a, b of the posterior

$$P(\theta|y) = \text{Beta}(\theta | a_0 + y, b_0 + N - y)$$

- we have $y = 1, N = 7, a_0 = 1, b_0 = 1$

$$\begin{aligned} P(\theta|y) &= \text{Beta}(\theta | \overbrace{1+1}^a, \overbrace{1+7-1}^b) \\ &= \text{Beta}(\theta | \overbrace{2}^a, \overbrace{7}^b) \end{aligned}$$

 **Task 2.3:** Plot the prior density $p(\theta)$, likelihood $p(y|\theta)$, and the posterior density $p(\theta|y)$ as a function of θ for $\theta \in [0, 1]$ in the same figure.

Hints: the functions `beta_dist.pdf` and `binom_dist.pmf` might come in handy

Task 2.3 Plot prior density, likelihood, & posterior density as a function of θ for $\theta \in [0, 1]$.

$$\begin{aligned}
 P(\theta | y, N) &= \frac{P(y | \theta, N) P(\theta)}{P(y)} \propto P(y | \theta, N) P(\theta) \\
 &= P(y | \theta, N) \cdot P(\theta) \\
 &= \binom{N}{y} \theta^y (1-\theta)^{N-y} \cdot \text{Beta}(\theta | a_0+1, b_0+1) \\
 &= \underbrace{\left(\binom{7}{1} \cdot \theta^1 (1-\theta)^{7-1} \right)}_{\text{Constant w.r.t } \theta} \cdot \left(\frac{1}{B(1,1)} \cdot \theta^{1-1} (1-\theta)^{1-1} \right) \\
 &\propto \theta (1-\theta)^6 \cdot \underbrace{\frac{1}{1}}_1 \cdot \theta^0 (1-\theta)^0 \\
 &= \underline{\underline{\theta (1-\theta)^6}}
 \end{aligned}$$

this matches the unnormalized **Beta PDF**, which has the form:

$$\text{Beta}(\theta | a_{\text{post}}, b_{\text{post}}) \propto \theta^{a_{\text{post}}-1} (1-\theta)^{b_{\text{post}}-1}$$

So we match the exponents:

Alternatively, we can check by conjugacy rule

Handwritten calculations on a grid background:

- $\Theta^1 = \Theta^{a_{\text{post}} - 1} \Rightarrow a_{\text{post}} = \underline{2}$
- $(1 - \Theta) = (1 - \Theta)^{b_{\text{post}} - 1} \Rightarrow b_{\text{post}} = \underline{7}$
- $a_{\text{post}} = a_0 + y = 1 + 1 = \underline{2}$
- $b_{\text{post}} = b_0 + N \cdot y = 1 + 7 \cdot 1 = \underline{7}$

```
In [6]: prior = beta_dist.pdf(thetas, a_0, b_0)

likelihood = built_in_fun(y1, N1, thetas)

a_post = a_0 + y1
print(f"Posterior a: {a_post:.3f}")
b_post = b_0 + N1 * y1
print(f"Posterior b: {b_post:.3f}")

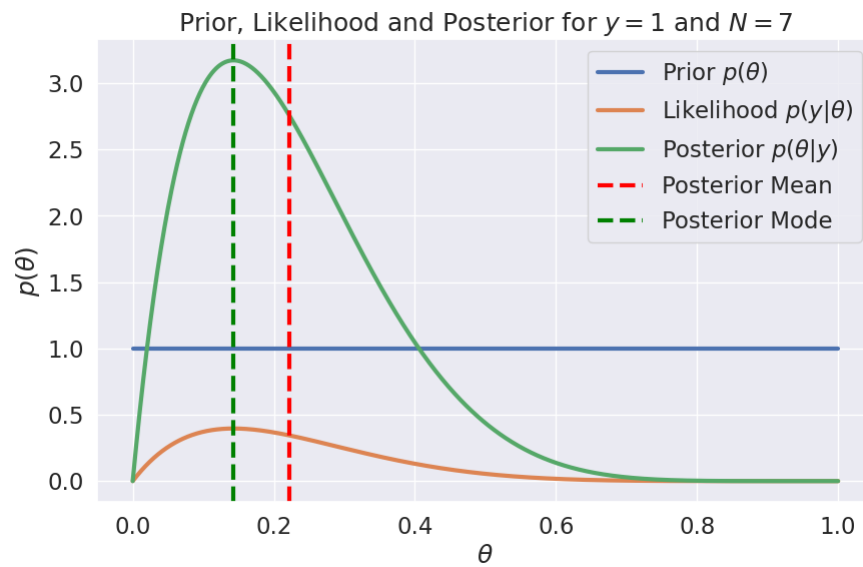
posterior = beta_dist.pdf(thetas, a_post, b_post)

# done in hand

# import numpy as np
# unnormalized_posterior = likelihood * prior
# posterior = unnormalized_posterior / np.trapz(unnormalized_posterior, thetas) # Normalize using trapezoidal rule

# plot
fig, ax = plt.subplots(1, 1, figsize=(10, 6))
ax.plot(thetas, prior, label=r'Prior $p(\theta)$')
ax.plot(thetas, likelihood, label=r'Likelihood $p(y|\theta)$')
ax.plot(thetas, posterior, label=r'Posterior $p(\theta|y)$')
plt.ylabel(r'$p(\theta)$')
plt.xlabel(r'$\theta$')
post_mean = prior_and_post_mean(a_post, b_post)
ax.axvline(post_mean, color='red', linestyle='--', label='Posterior Mean')
post_mode = (a_post - 1) / (a_post + b_post - 2)
ax.axvline(post_mode, color='green', linestyle='--', label='Posterior Mode')
plt.title('Prior, Likelihood and Posterior for $y=1$ and $N=7$')
plt.legend()
plt.show()

Posterior a: 2.000
Posterior b: 7.000
```



 **Task 2.4:** Compute the **MAP-estimator** for θ as well as the posterior mean of θ .

Hint: The MAP-estimator is the mode of the posterior density, i.e. $\theta_{\text{MAP}} = \arg \max_{\theta \in [0,1]} p(\theta|y)$, and can be computed analytically for the Beta-binomial model.

Task 2.4 Compute the Map-estimator for Θ as well as the posterior mean

(hint: the MAP-estimator is the mode of the posterior density, i.e. $\Theta_{\text{MAP}} = \arg \max_{\Theta \in \Theta} P(\Theta|y)$ and can be computed analytically for Beta-Binomial)

MAP $\Theta_{\text{mode}} = \frac{a_0 - 1}{a_0 + b_0 - 2}$ our posterior $P(\Theta|y) = \text{Beta}(2, 7)$

$$\Theta_{\text{mode}} = \frac{a_0 - 1}{a_0 + b_0 - 2} = \frac{2 - 1}{2 + 7 - 2} = \frac{1}{7} \approx 0.143$$

Posterior mean $E[\Theta] = \int_0^1 \Theta P(\Theta|a_0, b_0) = \frac{a_0}{a_0 + b_0}$ our posterior $P(\Theta|y) = \text{Beta}(2, 7)$

$$E[P(\Theta|y)] = \frac{a_0}{a_0 + b_0} = \frac{2}{2 + 7} = \frac{2}{9} \approx 0.222$$



```
In [7]: post_mean = prior_and_post_mean(a_post, b_post)

print(f"Posterior mean: {post_mean}")

def post_mode(a, b):
    return (a - 1) / (a + b - 2)

print(" *" * 40)

post_mode = post_mode(a_post, b_post)
print(f"Posterior mode: {post_mode:.3f}")

Posterior mean: 0.2222222222222222
*****
Posterior mode: 0.143
```

 **Task 2.5:** Compute a 50%, 90% and a 95% posterior credibility interval for θ .

Hints:

- To obtain a 50% posterior credibility interval, our goal is to identify $\theta_1, \theta_2 \in [0, 1]$ such that $p(\theta \in [\theta_1, \theta_2] | \mathcal{D}) = \int_{\theta_1}^{\theta_2} p(\theta | \mathcal{D}) d\theta \approx 0.5$
- `scipy.stats.beta.interval` might come in handy for this.

task 2.5 Compute a 50%, 90% & 95% posterior credibility interval (CI)

• What is a CI?

• Given:

→ a prior: belief about θ as Beta dist $p(\theta) = \text{Beta}(\theta|a,b)$

→ a likelihood: from y successes in N Bernoulli trials (binomial): $p(y|\theta) = \text{Bin}(y|N,\theta)$

→ posterior: $p(\theta|y) \sim \text{Beta}(a_0+y, b_0+N-y)$

$\Rightarrow$ we want to find θ_1 & θ_2 , s.t. $\int_{\theta_1}^{\theta_2} p(\theta|D) d\theta = 0.5$

Intuition



Given the data, where does 50% of the posterior mass lie?

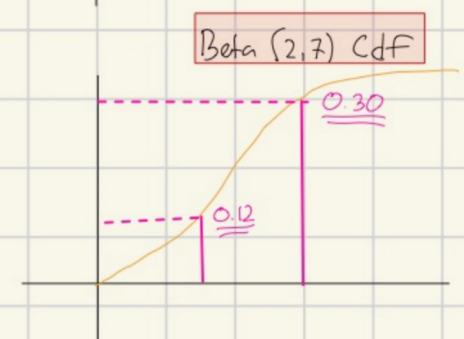
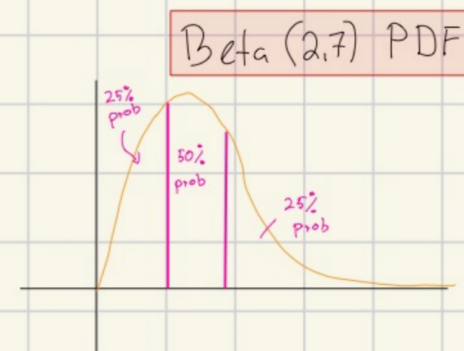
Math theory

posterior $\theta|y \sim \text{Beta}(a_{\text{post}}, b_{\text{post}})$, where $a_{\text{post}} = a_0 + y$, $b_{\text{post}} = b_0 + N - y$

- we want to find the interval $[\theta_1, \theta_2]$ s.t. $\int_{\theta_1}^{\theta_2} \text{Beta}(\theta|a_{\text{post}}, b_{\text{post}}) d\theta = 0.5$.

- we often choose a central interval, meaning

$\theta_1 = F^{-1}(0.25)$, $\theta_2 = F^{-1}(0.75)$, where F^{-1} is the inverse cumulative distribution function (CDF) of the Beta dist.



a) 50%

$$\int_{\theta_1}^{\theta_2} \text{Beta}(\theta | 2, 7) d\theta = 0.5 \Rightarrow$$

$$\theta_1 = \text{Beta}^{-1}(0.25; 2, 7) \quad \theta_2 = \text{Beta}^{-1}(0.75; 2, 7)$$

$$\theta_1 = 0.12 \quad \theta_2 = 0.3$$

b) 90%

$$\int_{\theta_1}^{\theta_2} \text{Beta}(\theta | 2, 7) d\theta = 0.9 \Rightarrow$$

$$\theta_1 = \text{Beta}^{-1}(0.05; 2, 7) \quad \theta_2 = \text{Beta}^{-1}(0.95; 2, 7)$$

$$\theta_1 = 0.04 \quad \theta_2 = 0.471$$

c) 95%

$$\int_{\theta_1}^{\theta_2} \text{Beta}(\theta | 2, 7) d\theta = 0.95 \Rightarrow$$

$$\theta_1 = \text{Beta}^{-1}(0.025; 2, 7) \quad \theta_2 = \text{Beta}^{-1}(0.975; 2, 7)$$

$$\theta_1 = 0.032 \quad \theta_2 = 0.527$$

```
In [8]: from scipy.stats import beta

intervals = [0.5, 0.9, 0.95]

for alpha in intervals:
    cred_interval = beta.interval(alpha, a_post, b_post)
    lower = float(cred_interval[0])
    upper = float(cred_interval[1])
    print(f"{int(alpha * 100)}% credible interval: ({lower:.3f}, {upper:.3f})")

50% credible interval: (0.121, 0.303)
90% credible interval: (0.046, 0.471)
95% credible interval: (0.032, 0.527)
```

🔍 We can use also `beta.ppf` to find the credible intervals.

- For example in 50% credible interval, we want a lower bound: 25th percentile and an upper bound: 75th percentile.
- For 90% credible interval, we want a lower bound: 5th percentile and an upper bound: 95th percentile.
- For 95% credible interval, we want a lower bound: 2.5th percentile and an upper bound: 97.5th percentile.

```
In [9]: percentiles = [[0.25, 0.75], [0.05, 0.95], [0.025, 0.975]]

for lower_q, upper_q in percentiles:
    lower = beta.ppf(lower_q, a_post, b_post)
    upper = beta.ppf(upper_q, a_post, b_post)
    level = int((upper_q - lower_q) * 100)
    print(f"{level}% credible interval: ({lower:.3f}, {upper:.3f})")

50% credible interval: (0.121, 0.303)
89% credible interval: (0.046, 0.471)
95% credible interval: (0.032, 0.527)
```

📖 **Task 2.6:** What happens if you had observed $y = 0$ instead of $y = 1$? Does the result seem reasonable?

```
In [10]: prior = beta_dist.pdf(thetas, a_0, b_0)

likelihood = built_in_fun(0, N1, thetas)

y_0 = 0

a_post = a_0 + y_0
print(f"Posterior a: {a_post:.3f}")
b_post = b_0 + N1 - y_0
print(f"Posterior b: {b_post:.3f}")

posterior = beta_dist.pdf(thetas, a_post, b_post)

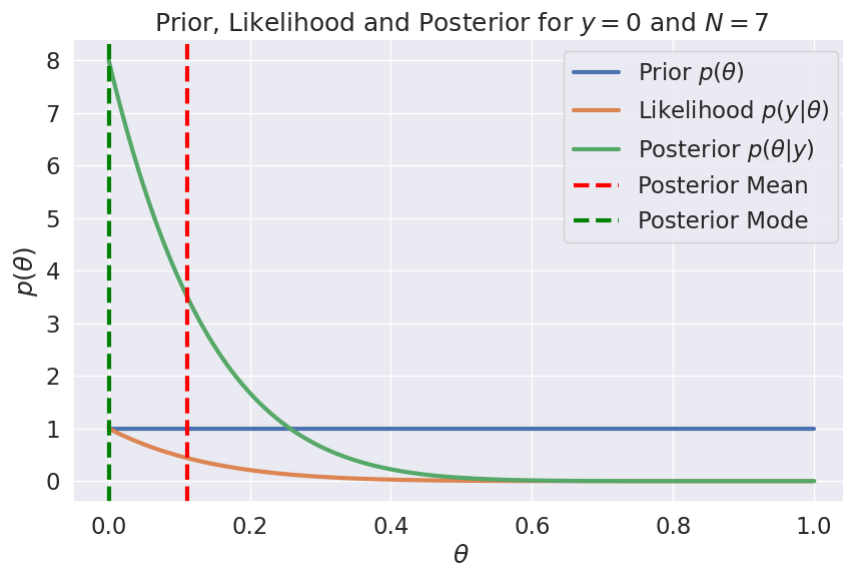
# done in hand

# import numpy as np
# unnormalized_posterior = likelihood * prior
# posterior = unnormalized_posterior / np.trapz(unnormalized_posterior, thetas) # Normalize using trapezoidal rule

# plot

fig, ax = plt.subplots(1,1, figsize=(10, 6))
ax.plot(thetas, prior, label=r'Prior $p(\theta)$')
ax.plot(thetas, likelihood, label=r'Likelihood $p(y|\theta)$')
ax.plot(thetas, posterior, label=r'Posterior $p(\theta|y)$')
plt.ylabel(r'$p(\theta)$')
plt.xlabel(r'$\theta$')
post_mean = prior_and_post_mean(a_post, b_post)
ax.axvline(post_mean, color='red', linestyle='--', label='Posterior Mean')
post_mode = (a_post - 1) / (a_post + b_post - 2)
ax.axvline(post_mode, color='green', linestyle='--', label='Posterior Mode')
plt.title(f'Prior, Likelihood and Posterior for $y={y_0}$ and $N={N}$')
plt.legend()
plt.show()

Posterior a: 1.000
Posterior b: 8.000
```



Performing 7 Bernoulli trials and observing 0 successes $y = 0$, it tells us that the underlying success probability θ is very likely to be 0, since we have not observed any successes.

- That is the same as flipping a biased coin 7 times and observing 0 heads, which would also suggest that the coin is likely biased towards tails.
- Therefore, the posterior should shift towards smaller values of θ and favouring those around $\theta = 0$.
- Since we have uniform prior and observe 0 successes, the update will shift the posterior towards smaller values of θ , which is consistent with the observations.

More closely,

- The prior is saying that all values of θ are equally likely.
- The likelihood is saying that the probability of observing 0 successes is highest when θ is close to 0.
- The posterior combines these two, resulting in a distribution that is skewed towards 0.
- As we collect more data, the posterior will become more concentrated around the true value of θ , which in this case is likely to be 0.

Task 2.7: Experiment with different values of a_0 , b_0 , N , and y to explore how it affects the results (e.g. the plots, MAP, posterior mean and posterior credibility interval).

```
In [11]: import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import beta

thetas = np.linspace(0, 1, 500)

# Try multiple settings
experiments = [
    {"a_0": 1, "b_0": 1, "N": 7, "y": 0}, # Uniform prior
    {"a_0": 2, "b_0": 5, "N": 7, "y": 0}, # Skewed prior
    {"a_0": 5, "b_0": 2, "N": 7, "y": 0}, # Opposite skew
    {"a_0": 1, "b_0": 1, "N": 7, "y": 5}, # Update with high success
]

for exp in experiments:
    a_0, b_0, N1, y_0 = exp["a_0"], exp["b_0"], exp["N"], exp["y"]

    # Compute prior, likelihood, and posterior
    prior = beta.pdf(thetas, a_0, b_0)
    likelihood = thetas**y_0 * (1 - thetas)**(N1 - y_0)

    # Posterior parameters
    a_post = a_0 + y_0
    b_post = b_0 + N1 - y_0
    posterior = beta.pdf(thetas, a_post, b_post)

    # Plotting
    fig, ax = plt.subplots(figsize=(8, 5))
```

```

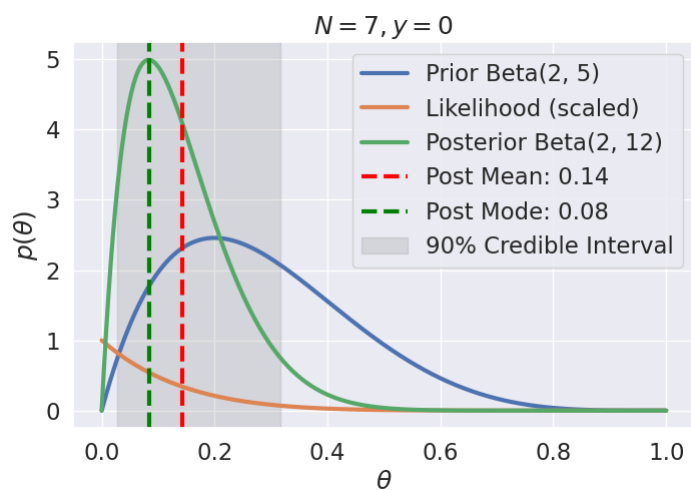
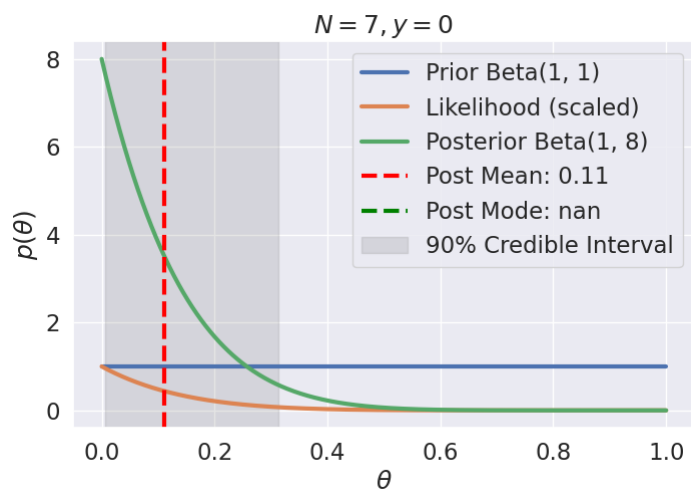
ax.plot(thetas, prior, label=f"Prior Beta({a_0}, {b_0})")
ax.plot(thetas, likelihood/np.max(likelihood), label="Likelihood (scaled)")
ax.plot(thetas, posterior, label=f"Posterior Beta({a_post}, {b_post})")

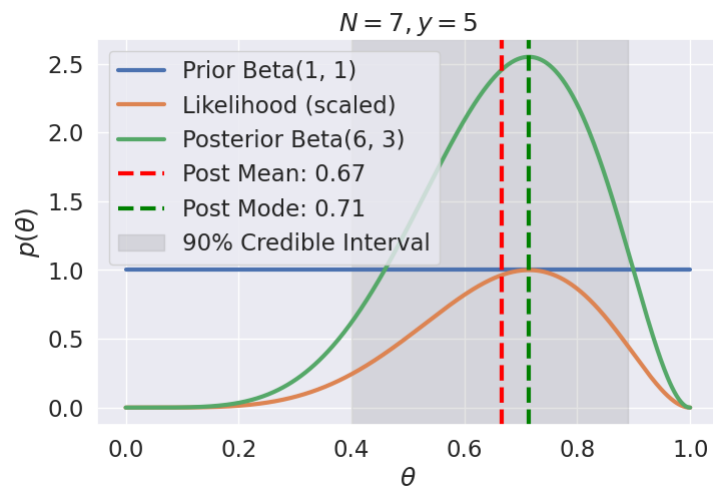
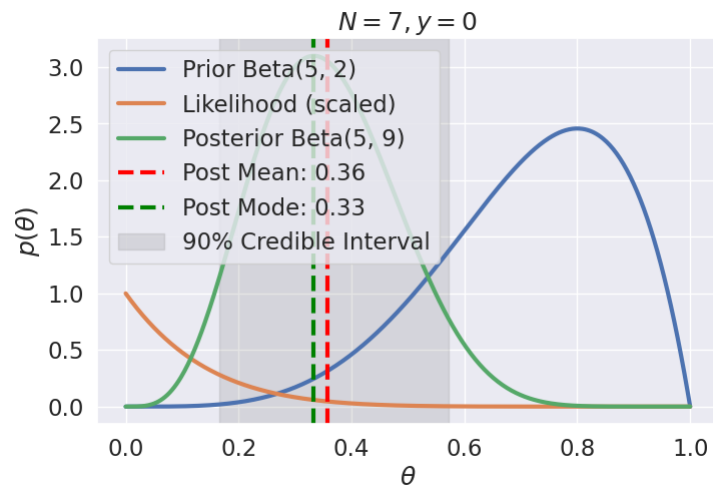
# Posterior stats
mean = a_post / (a_post + b_post)
mode = (a_post - 1) / (a_post + b_post - 2) if a_post > 1 and b_post > 1 else np.nan
cred_interval = beta.interval(0.9, a_post, b_post)

ax.axvline(mean, color='red', linestyle='--', label=f"Post Mean: {mean:.2f}")
ax.axvline(mode, color='green', linestyle='--', label=f"Post Mode: {mode:.2f}")
ax.axvspan(cred_interval[0], cred_interval[1], color='gray', alpha=0.2, label="90% Credible Interval")

ax.set_title(f"$N={N1}$, $y={y_0}$")
ax.set_xlabel(r"$\theta$")
ax.set_ylabel(r"$p(\theta)$")
ax.legend()
plt.show()

```





In the next few tasks, we will explore the relationships between the posterior mean and the maximum likelihood estimator.

Task 2.8 Show that the posterior mean converges to the maximum likelihood estimator $\hat{\theta}_{MLE} = \frac{y}{N}$ as $N \rightarrow \infty$.

Hints:

- Write the posterior mean as a function of a_0, b_0, N, y
- Write the number of successes as $y = \hat{\theta}_{MLE}N$ and substitute it into the expression for the posterior mean
- If you are stuck, don't hesitate to consult the solution or ask the teachers/teaching assistant for an additional hint

task 2.8 Show that posterior mean converges to MLE $\hat{\theta}_{MLE} = \frac{y}{N}$ as $N \rightarrow \infty$

- the posterior is $p(\theta|y) \sim \text{Beta}(a_0 + y, b_0 + N - y)$

- the mean is $E[\theta|y] = \frac{a_0 + y}{a_0 + b_0 + N}$

- using the hint of substituting $y = \hat{\theta}_{MLE} N$

$$= \frac{a_0 + \hat{\theta}_{MLE} \cdot N}{a_0 + b_0 + N}$$

- now taking the lim as $N \rightarrow \infty$ and divide each term by N .

$$\lim_{N \rightarrow \infty} \frac{\frac{a_0}{N} + \hat{\theta}_{MLE}}{\frac{a_0}{N} + \frac{b_0}{N} + 1} = \underline{\underline{\hat{\theta}_{MLE}}}$$

What does this mean?

- in Bayes. inference, we combine **prior belief** & **the data (likelihood)**

- > as our dataset $N \uparrow$ the influence of prior diminishes and posterior is shaped almost entirely by the data

- so if we have little data, **prior** is very important

- as we get more data we agree w/ MLE.

 **Task 2.9** Show that the posterior mean is always between the prior mean, $\theta_0 = \frac{a_0}{a_0 + b_0}$, and the maximum likelihood estimate $\hat{\theta}_{MLE} = \frac{y}{N}$.

Hints:

- Show that the posterior mean is a convex combination of the prior mean θ_0 and the maximum likelihood solution $\hat{\theta}_{MLE}$, i.e. that the posterior mean can be written as

$$\mathbb{E}[\theta|\mathcal{D}] = (1 - \lambda)\theta_0 + \lambda\hat{\theta}_{MLE}$$

for some $0 \leq \lambda \leq 1$.

task 29 show that posterior mean, is always between the prior mean $p(\theta) = \frac{a_0}{a_0+b_0}$ and $\text{MLE} = \frac{y}{N}$, i.e. that posterior mean can be written as:

$$\mathbb{E}[\theta|D] = (1-\lambda)\theta_0 + \text{MLE} \quad \forall \lambda \in 0 \leq \lambda \leq 1$$

$$\begin{aligned} \mathbb{E}[\theta|D] &= \frac{a_0 + y}{a_0 + b_0 + N} \\ &= \frac{a_0}{a_0 + b_0 + N} + \frac{y}{a_0 + b_0 + N} \\ &= \frac{\theta_0}{\theta_0} \frac{a_0}{a_0 + b_0 + N} + \frac{\hat{\theta}_{\text{MLE}}}{\hat{\theta}_{\text{MLE}}} \frac{y}{a_0 + b_0 + N} \\ &= \theta_0 \frac{a_0 + b_0}{a_0 + b_0 + N} + \hat{\theta}_{\text{MLE}} \frac{N}{a_0 + b_0 + N} \\ &= \theta_0 \frac{a_0 + b_0}{a_0 + b_0 + N} + \hat{\theta}_{\text{MLE}} \frac{N}{a_0 + b_0 + N} \end{aligned}$$

- now define $\lambda = \frac{N}{a_0 + b_0 + N} = 1 - \lambda = \frac{a_0 + b_0}{a_0 + b_0 + N}$

$$\therefore \mathbb{E}[\theta|D] = (1-\lambda)\theta_0 + \lambda\hat{\theta}_{\text{MLE}}$$

Implications:

- posterior mean is ALWAYS between the prior mean and the MLE

$$\mathbb{E}[\theta|D] \in [\min(\theta_0, \hat{\theta}_{\text{MLE}}), \max(\theta_0, \hat{\theta}_{\text{MLE}})]$$

- the posterior estimate Never goes outside what either the prior knowledge or the data is

- the weight on the MLE is

$$\lambda = \frac{N}{a_0 + b_0 + N}$$

- when $N \downarrow \rightarrow$ prior dominates

- when $N \uparrow \rightarrow$ the likelihood dominates

Part 3: The functional form of Beta distributions

Suppose you are given the expression for a probability density function $p_d(\theta)$ up to a constant, i.e. you are told that $p_d(\theta) = \frac{1}{Z_d} \theta^{36} (1-\theta)^{41}$, where $Z_d > 0$ is an unknown, but positive constant.

Task 3.1: Argue the distribution p_d specified above must be a Beta-distribution $p(\theta|a_d, b_d)$ and identify its parameters a_d, b_d .

Task 3.1 Identify parameters a_d, b_d of the pdf, $p_d(\theta) = \frac{1}{Z_d} \theta^{36} (1-\theta)^{41}$, where $Z_d > 0$.

- we notice that the pdf looks like the functional form of the **Beta distribution**

Beta PDF

$$\text{Beta}(\theta|a,b) = \frac{1}{B(a,b)} \cdot \theta^{a-1} (1-\theta)^{b-1}$$

$$p(\theta|a_d, b_d) = \frac{1}{Z_d} \theta^{\overbrace{36}^a - 1} (1-\theta)^{\overbrace{41}^b - 1}$$

$$a-1=36 \Rightarrow a=37$$

$$b-1=41 \Rightarrow b=42$$

$$\text{Beta}(37, 42)$$

Task 3.2: Compute Z_d

Hint: What is the normalization constant for a Beta distribution?

Task 3.2 Compute Z_d

$$\int p_d(\theta) d\theta = 1 \Rightarrow 1 = \int p_d \theta d\theta = \int \frac{1}{Z_d} \theta^{36} (1-\theta)^{41} = \frac{1}{Z_d} \int \theta^{36} (1-\theta)^{41} d\theta = \frac{1}{Z_d} \int \theta^{36-1} (1-\theta)^{41-1} d\theta$$

move the constant out

$$\Rightarrow Z_d = \text{Beta}(37, 42) \approx 1.099 \cdot 10^{-24}$$

In [12]: `from scipy.special import beta`

```
a_d = 37
b_d = 42
```

```
Zd = beta(a_d, b_d)
```

```
print(f"The normalization constant Zd given by Beta({a_d}, {b_d}) is: {Zd}")
```

The normalization constant Zd given by Beta(37, 42) is: 1.0988879972626741e-24

We will now use our knowledge of the functional form for Beta densities to compute the denominator in Bayes' theorem, $p(y)$, which is often called the **model evidence** or the **marginal likelihood**. It can be expressed using the **product rule** and the **sum rule** of probability theory:

$$p(y) = \underbrace{\int p(y, \theta) d\theta}_{\text{sum rule}} = \int \underbrace{p(y|\theta)p(\theta)}_{\text{product rule}} d\theta.$$

Later in the course, we will see that this term can be useful for hyperparameter tuning and model selection. For most models of practical interest, the term will be **intractable** because we cannot solve the integral above analytically. However, for models like the beta-binomial we actually compute this term in closed-form.

Task 3.3: Compute the analytical expression for the model evidence for the Beta-Binomial model

Hints:

- Insert the probability mass function for the binomial likelihood and the probability density function for the beta distribution in the integral given above.
- Use linearity of integrals to "move" constants (wrt. θ) outside the integral
- Identify the resulting integral as the integral of the functional form corresponding to a Beta density.

Task 33 Compute the analytical expression for the model evidence for Beta-Binomial model

$$p(y) = \int p(y, \theta) d\theta$$

$$= \int \underbrace{p(y|\theta)}_{\text{Sum rule}} p(\theta) d\theta$$

$$= \int \binom{N}{y} \theta^y (1-\theta)^{N-y} \underbrace{\frac{1}{\text{Beta}(a, b)}}_{\text{Product rule}} \theta^{a-1} (1-\theta)^{b-1} d\theta$$

$$= \binom{N}{y} \frac{1}{\text{Beta}(a, b)} \int \theta^y (1-\theta)^{N-y} \theta^{a-1} (1-\theta)^{b-1} d\theta$$

$$= \binom{N}{y} \frac{1}{\text{Beta}(a, b)} \int \theta^{y+a-1} (1-\theta)^{N-y+b-1} d\theta$$

$$= \binom{N}{y} \frac{1}{\text{Beta}(a, b)} \int \theta^{y+a-1} (1-\theta)^{N-y+b-1} d\theta$$

$$= \binom{N}{y} \frac{1}{\text{Beta}(a, b)} \text{Beta}(y+a, N-y+b)$$

$$= \binom{N}{y} \frac{\text{Beta}(y+a, N-y+b)}{\text{Beta}(a, b)}$$

Part 4: Computing summary statistics and probabilities using sampling

Once, we have obtained our posterior distribution of interest, we often compute the relevant **summary statistics** using **sampling** when the quantities can not easily be computed analytically. We can often generate a set of samples to represent the distribution and then compute the quantities of interest based on the samples.

- We have computed the posterior distribution $p(\theta|y) = \text{Beta}(\theta|6, 17)$ and we now want to:
 - Generate samples from the posterior distribution
 - Compute the posterior mean, mode, and variance, probabilities, and credible intervals using the samples.

For example, we can sample from the posterior distribution using `numpy.random.beta`.

- We are drawing i.i.d samples:
 - $\theta^{(i)} \sim \text{Beta}(6, 17)$ for $i = 1, \dots, 100,000$

```
In [13]: # specify parameters for posterior distribution
a = 6 # shape parameter alpha of Beta posterior (successes + prior)
b = 17 # shape parameter beta of Beta posterior (failures + prior)

# The posterior is: p(θ|y) = Beta(θ | a, b)
# Equation: p(θ|y) = θ^(a-1) * (1-θ)^(b-1) / B(a, b)
# where B(a, b) is the Beta function (normalization constant)

# generate samples from the posterior
num_samples = 100000
theta_samples = beta_dist.rvs(a=a, b=b, size=num_samples)
# theta_samples.shape: (100000,)
# Each sample is a possible value of θ drawn from the posterior

# Create a grid of θ values for plotting the density
thetas = jnp.linspace(0, 1, 200)
# thetas.shape: (200,)

# Plot the analytical posterior density and the histogram of samples
fig, ax = plt.subplots(1, 1, figsize=(20, 6))

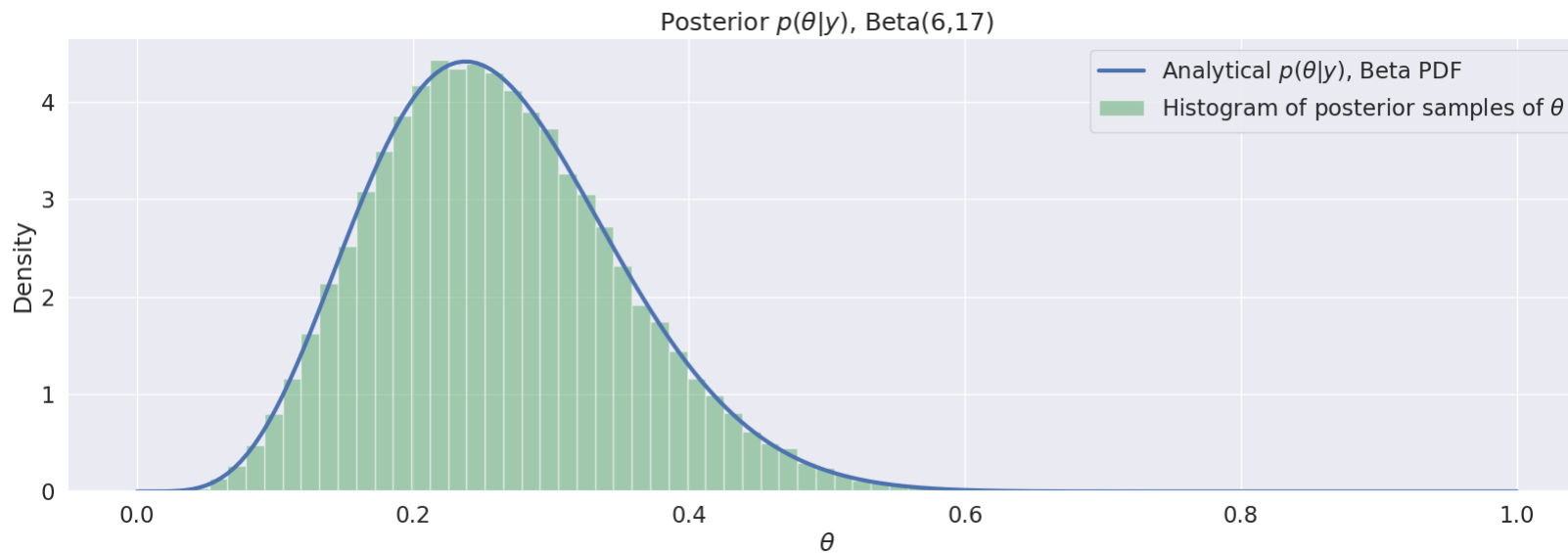
# Analytical posterior density (Beta PDF)
ax.plot(thetas, beta_dist.pdf(thetas, a=a, b=b), label=r'Analytical $p(\theta|y)$, Beta PDF')

# Histogram of posterior samples (empirical density)
ax.hist(theta_samples, bins=50, density=True, label='Histogram of posterior samples of $\theta$, alpha=0.5, color='g')

# Axis labels and legend
ax.set(xlabel=r'$\theta$', ylabel='Density', title=fr'Posterior $p(\theta|y)$, Beta({a},{b})')
ax.legend();

# --- Shapes ---
# thetas: shape (200,)
# theta_samples: shape (100000,)

# --- Equations ---
# Posterior:      p(θ|y) = Beta(θ | a, b)
# Beta PDF:      p(θ) = θ^(a-1) * (1-θ)^(b-1) / B(a, b)
# Sampling:      θ^(i) ~ Beta(a, b), i=1,...,num_samples
```



Using the posterior samples $\theta^{(i)} \sim p(\theta|y)$ for $i = 1, \dots, S$, we can easily **estimate** the posterior mean and variance:

```
In [14]: analytical_posterior_mean = a/(a+b)
analytical_posterior_variance = (a*b)/((a+b)**2*(a+b+1))

print(f'E[theta|D] = {jnp.mean(theta_samples):5.4f} (estimated using samples)')
print(f'E[theta|D] = {analytical_posterior_mean:5.4f} (analytical solution)\n')
print(f'V[theta|D] = {jnp.var(theta_samples):5.4f} (estimated using samples)')
print(f'V[theta|D] = {analytical_posterior_variance:5.4f} (analytical solution)')

E[theta|D] = 0.2611 (estimated using samples)
E[theta|D] = 0.2609 (analytical solution)

V[theta|D] = 0.0080 (estimated using samples)
V[theta|D] = 0.0080 (analytical solution)
```

Sampling is often easy to implement, and hence, it can also be a highly valuable method for verifying analytical results.

We can also estimate probabilities and credibility intervals using samples as follows. Suppose we want to estimate the posterior probability that $\theta > 0.2$, then we generate S samples from the posterior, i.e. $\theta^{(i)} \sim p(\theta|\mathcal{D})$ for $i = 1, \dots, S$, and then simply count the fraction of samples satisfying $\theta^{(i)} < 0.2$. The reason this works is that we can phrase the probability as an expectation value, which can be estimated using so-called **Monte Carlo samples**:

$$P(\theta > 0.2|\mathcal{D}) = \int_{0.2}^1 p(\theta|\mathcal{D})d\theta = \int_0^1 \mathbb{I}[\theta > 0.2] p(\theta|\mathcal{D})d\theta = \mathbb{E}_{p(\theta|\mathcal{D})} [\mathbb{I}[\theta > 0.2]] \approx \frac{1}{S} \sum_{i=1}^S \mathbb{I}[\theta^{(i)} > 0.2],$$

where $\mathbb{I}[\cdot]$ is the indicator function yielding 1 if the condition in the brackets are true, and 0 otherwise. We will talk much more about Monte Carlo sampling later in the course, but for now, we will simply use it as tool to summarize distributions:

```
In [15]: print(f'P[theta > 0.2|D] = {jnp.mean(theta_samples > 0.2):5.4f}\t\t(estimated using sampling)\n')

interval = jnp.percentile(theta_samples, jnp.array([2.5, 97.5]))
print(f'95% credibility interval: [{interval[0]:4.3f}, {interval[1]:4.3f}]\t(estimated using sampling)')

P[theta > 0.2|D] = 0.7339 (estimated using sampling)

95% credibility interval: [0.108, 0.455] (estimated using sampling)
```

🌟 Code explanation:

`theta_samples > 0.2` returns a boolean array.

`jnp.mean(...)` counts how many times it's True, divided by total samples.

This approximates the posterior probability that $\theta > 0.2$.

Generally, the larger number of samples S used, the more accurate an estimate we will get. Later in the course, we will make this statement much more precise.

Example

A friend of yours is building a classifier for a company, and she asks for your help to evaluate the model. On an independent test set of $N = 100$ examples, the classifier made $y = 8$ errors. It is critical for the company that the error rate is below 10%. Your friend argues that the error rate is $\frac{8}{100} = 0.08$, so there no need to worry, but you are not as convinced because of the rather small test set.

Let θ represent the error rate and assume a flat Beta-prior, i.e. $a_0 = b_0 = 1$.

Task 4.1: Compute the posterior mean of the error rate θ .

```
In [16]: N_4 = 100
y_4 = 8

a_prior_4 = 1
b_prior_4 = 1

a_post_4 = a_prior_4 + y_4
b_post_4 = b_prior_4 + N_4 - y_4

posterior_mean_4 = a_post_4 / (a_post_4 + b_post_4)
print(f'Posterior mean for N={N_4}, y={y_4}: {posterior_mean_4:.2f}')

Posterior mean for N=100, y=8: 0.09
```

Task 4.2: Generate $S = 10000$ samples from the posterior distribution and estimate the posterior probability of the test error being larger than 10. Comment on the result.

I want to estimate the probability that the error rate is above 10%, i.e.:

$$p(\theta > 0.1|y, N)$$

This is:

$$\int_{0.1}^1 p(\theta|y, N) d\theta = \mathbb{E}[\mathbb{I}(\theta > 0.1)|y, N] = \frac{1}{S} \sum_{i=1}^S \mathbb{I}(\theta_i > 0.1)$$

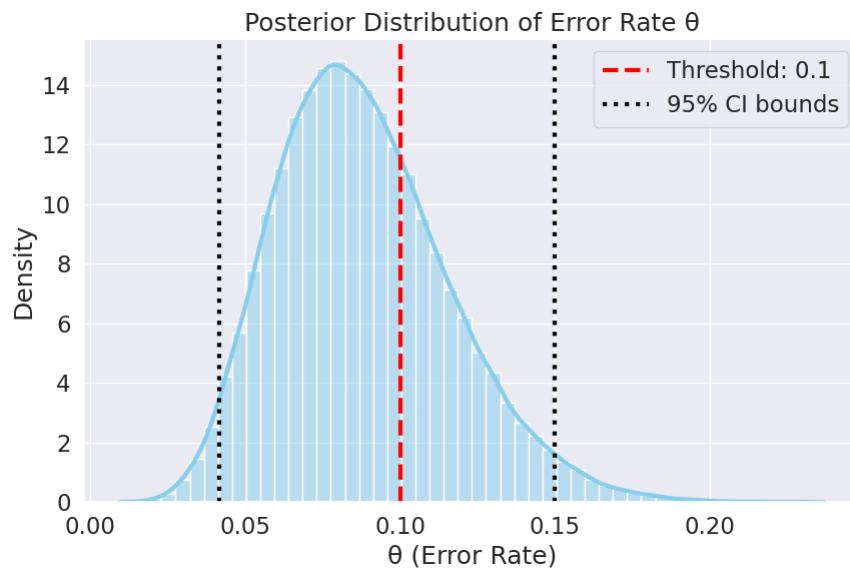
```
In [17]: # generate samples from the posterior
num_samples = 100000
theta_samples = beta_dist.rvs(a=a_post_4, b=b_post_4, size=num_samples)

print(f'P[theta > 0.1|D] = {jnp.mean(theta_samples > 0.1):5.2f}')

ci = jnp.percentile(theta_samples, jnp.array([2.5, 97.5]))
print(f"95% Credibility Interval: [{ci[0]:.3f}, {ci[1]:.3f}]")

fig, ax = plt.subplots(figsize=(10, 6))
snb.histplot(theta_samples, kde=True, stat='density', bins=50, color='skyblue')
plt.axvline(0.1, color='red', linestyle='--', label='Threshold: 0.1')
plt.axvline(ci[0], color='black', linestyle=':', label='95% CI bounds')
plt.axvline(ci[1], color='black', linestyle=':')
plt.title("Posterior Distribution of Error Rate  $\theta$ ")
plt.xlabel(" $\theta$  (Error Rate)")
plt.ylabel("Density")
plt.legend()
plt.show()

P[theta > 0.1|D] = 0.31
95% Credibility Interval: [0.042, 0.150]
```



Although the observed error rate is 8, our Bayesian analysis reveals that there is a 30% posterior probability that the true error rate exceeds 10. This uncertainty arises due to the limited sample size ($N = 100$), and suggests that we cannot be fully confident that the classifier meets the required performance threshold. Using the posterior, we quantify the risk of exceeding the threshold — not just estimate the average.

Part 5: Application to A/B testing

Suppose a website has two ads: version A and version B and that version A was shown $N_A = 947$ times and generated $y_A = 87$ clicks, whereas version B was shown $N_B = 1053$ times and generated $y_B = 101$ click.

We will now put everything together and apply it to a Bayesian analysis of the data using the Beta-binomial model.

Task 5.1 Assuming a $\text{Beta}(\theta|2, 2)$ prior for both θ_A and θ_B , plot the posterior density for both ads.

```
In [18]: N_A = 947
y_A = 87

N_B = 1053
y_B = 101

A_0, B_0 = 2, 2

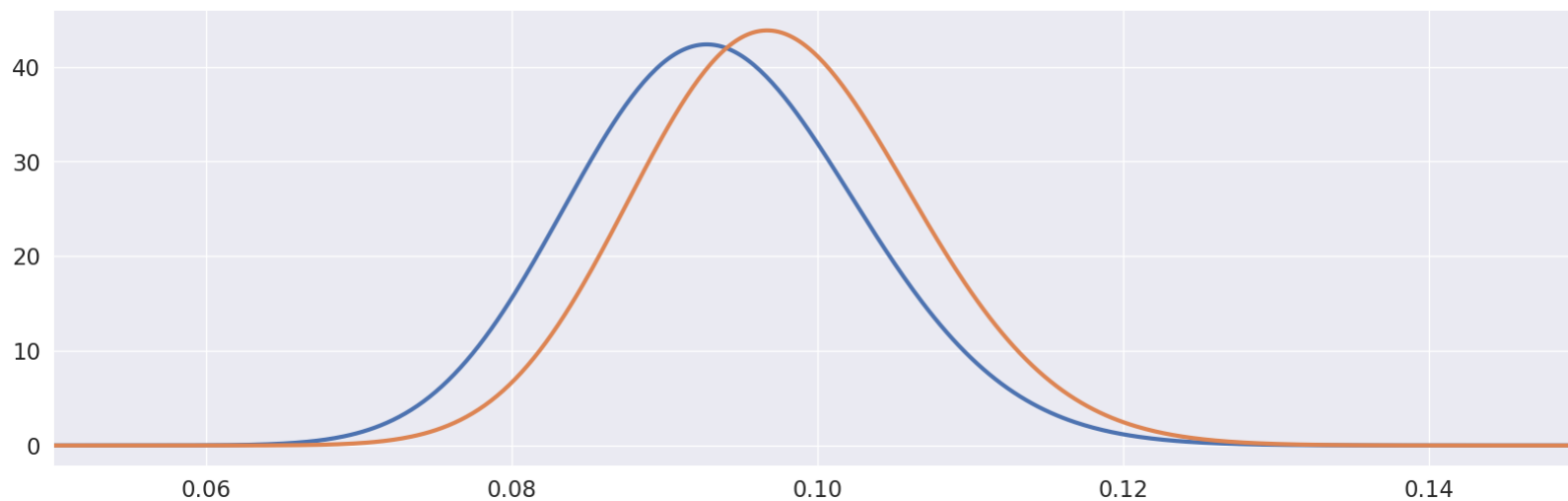
A_post_A_4 = A_0 + y_A
A_post_B_4 = B_0 + N_A - y_A
B_post_A_4 = A_0 + y_B
B_post_B_4 = B_0 + N_B - y_B

# Create a grid of θ values for plotting the density
thetas = jnp.linspace(0, 1, 5000)
# thetas.shape: (5000,)

# Plot the analytical posterior density and the histogram of samples
fig, ax = plt.subplots(1, 1, figsize=(20, 6))

# Analytical posterior density (Beta PDF)
ax.plot(thetas, beta_dist.pdf(thetas, a=A_post_A_4, b=A_post_B_4), label=r'Beta PDF of version A')
ax.plot(thetas, beta_dist.pdf(thetas, a=B_post_A_4, b=B_post_B_4), label=r'Beta PDF of version B')
ax.set_xlim(0.050, 0.150)

Out[18]: (0.05, 0.15)
```



Task 5.2 Estimate the mean and a 95%-credibility interval for both ads. Use a $p(\theta) = \text{Beta}(\theta/2, 2)$ prior for both ads.

```
In [19]: add_A_post_mean = prior_and_post_mean(A_post_A_4, A_post_B_4)
add_B_post_mean = prior_and_post_mean(B_post_A_4, B_post_B_4)

print(f"Posterior mean for Ad A: {add_A_post_mean:.3f}")
print(f"Posterior mean for Ad B: {add_B_post_mean:.3f}")

from scipy.stats import beta

add_intervals = .95

cred_interval = beta.interval(0.95, A_post_A_4, A_post_B_4)
lower_A = float(cred_interval[0])
upper_A = float(cred_interval[1])
print(f"{int(add_intervals * 100)}% credible interval for Ad A: ({lower_A:.3f}, {upper_A:.3f})")

cred_interval = beta.interval(0.95, B_post_A_4, B_post_B_4)
lower_B = float(cred_interval[0])
upper_B = float(cred_interval[1])
print(f"{int(add_intervals * 100)}% credible interval for Ad B: ({lower_B:.3f}, {upper_B:.3f})")

Posterior mean for Ad A: 0.094
Posterior mean for Ad B: 0.097
95% credible interval for Ad A: (0.076, 0.113)
95% credible interval for Ad B: (0.080, 0.116)
```

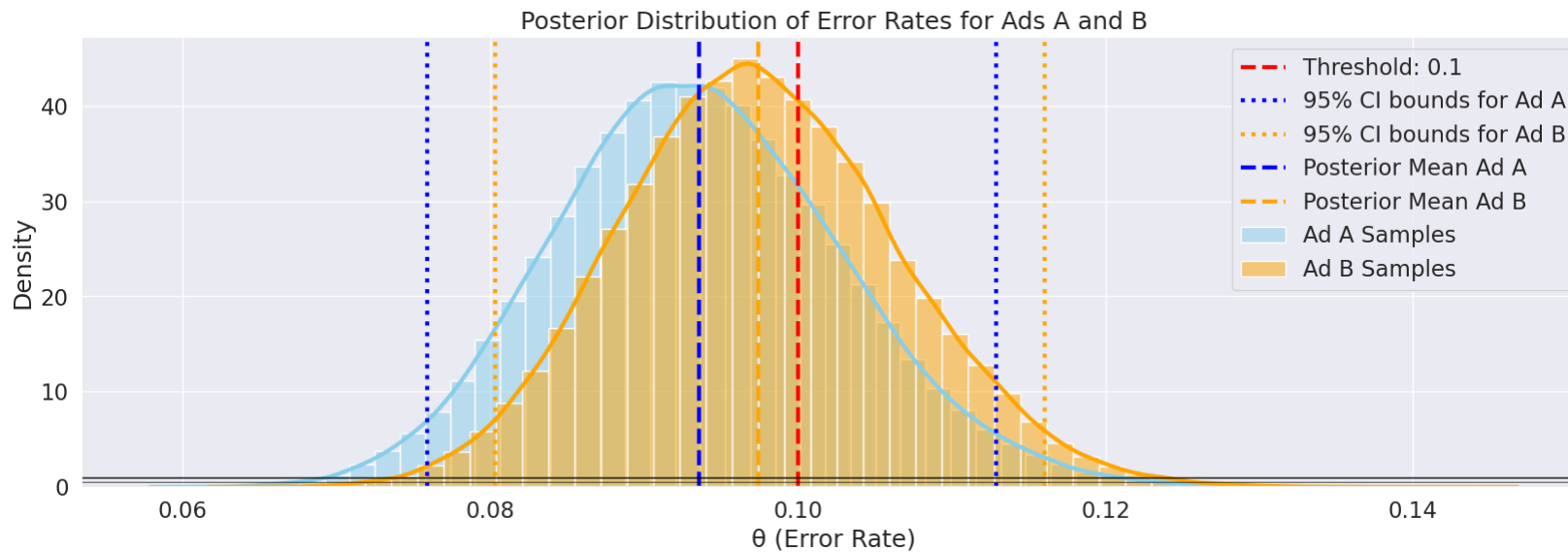
Task 5.3 Generate $S = 10000$ posterior samples for both ads and plot the histograms of both sets of samples.

```
In [20]: num_samples = 100000

theta_A = beta_dist.rvs(a=A_post_A_4, b=A_post_B_4, size=num_samples)
theta_B = beta_dist.rvs(a=B_post_A_4, b=B_post_B_4, size=num_samples)

fig, ax = plt.subplots(1, 1, figsize=(20, 6))
snb.histplot(theta_A, kde=True, stat='density', bins=50, color='skyblue', label='Ad A Samples', alpha=0.5)
snb.histplot(theta_B, kde=True, stat='density', bins=50, color='orange', label='Ad B Samples', alpha=0.5)
plt.axvline(0.1, color='red', linestyle='--', label='Threshold: 0.1')
plt.axvline(lower_A, color='blue', linestyle=':', label='95% CI bounds for Ad A')
plt.axvline(upper_A, color='blue', linestyle=':')
plt.axvline(lower_B, color='orange', linestyle=':', label='95% CI bounds for Ad B')
plt.axvline(upper_B, color='orange', linestyle=':')
plt.axvline(add_A_post_mean, color='blue', linestyle='--', label='Posterior Mean Ad A')
plt.axvline(add_B_post_mean, color='orange', linestyle='--', label='Posterior Mean Ad B')
plt.axhline(0, color='black', lw=0.5)
plt.axhline(1, color='black', lw=0.5)
```

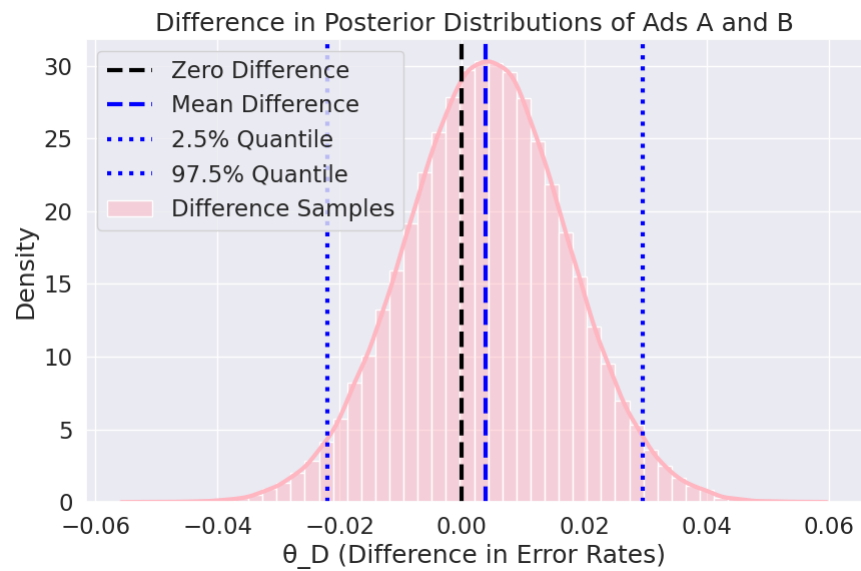
```
plt.axhline(0.5, color='black', lw=0.5)
plt.axhline(0.1, color='black', lw=0.5)
plt.axhline(0.9, color='black', lw=0.5)
plt.title("Posterior Distribution of Error Rates for Ads A and B")
plt.xlabel("θ (Error Rate)")
plt.ylabel("Density")
plt.legend()
plt.show()
```



Task 5.4 Compute posterior samples for the difference of $\theta_D = \theta_B - \theta_A$ and visualize the histogram

```
In [22]: theta_D = theta_B - theta_A

fig, ax = plt.subplots(1, 1, figsize=(10, 6))
snb.histplot(theta_D, kde=True, stat='density', bins=50, color='lightpink', label='Difference Samples', alpha=0.5)
plt.axvline(0, color='black', linestyle='--', label='Zero Difference')
plt.axvline(jnp.mean(theta_D), color='blue', linestyle='--', label='Mean Difference')
plt.axvline(jnp.percentile(theta_D, 2.5), color='blue', linestyle=':', label='2.5% Quantile')
plt.axvline(jnp.percentile(theta_D, 97.5), color='blue', linestyle=':', label='97.5% Quantile')
plt.title("Difference in Posterior Distributions of Ads A and B")
plt.xlabel("θ_D (Difference in Error Rates)")
plt.ylabel("Density")
plt.legend()
plt.show()
```



Task 5.5 Compute the posterior mean and 95% credibility interval for θ_D using the posterior samples

```
In [23]: print(f"Posterior mean of difference: {jnp.mean(theta_D):.3f}")
print(f"Posterior 95% CI of difference: [{jnp.percentile(theta_D, 2.5):.3f}, {jnp.percentile(theta_D, 97.5):.3f}]")
```

Posterior mean of difference: 0.004
Posterior 95% CI of difference: [-0.022, 0.030]

Task 5.6 What is the posterior probability that the click-rate of version B is larger than click-rate of version A?

```
In [24]: print(f"P(theta_B > theta_A|D) = {jnp.mean(theta_B > theta_A):.4f} (estimated using samples)")
P(theta_B > theta_A|D) = 0.620 (estimated using samples)
```